

Universidad Privada Domingo Savio

Tecnologia Web II

Entrega 8A

Schema SQL, Politicas RLS y Navegacion

FASE 4: Horarios, Asistencias y Configuración de Evaluación

Objetivos de Aprendizaje

- Comprender la relacion entre las tablas nuevas (horarios, asistencias, configuracion_evaluacion) y el esquema existente del MVP.
- Ejecutar DDL (Data Definition Language) para crear 3 tablas con CHECK constraints, indices y triggers en Supabase.
- Configurar politicas RLS (Row Level Security) diferenciadas por rol para cada tabla nueva.
- Actualizar la navegacion del sidebar para incluir las nuevas secciones de Fase 4.
- Crear paginas placeholder para las rutas de horarios, asistencias y evaluacion.

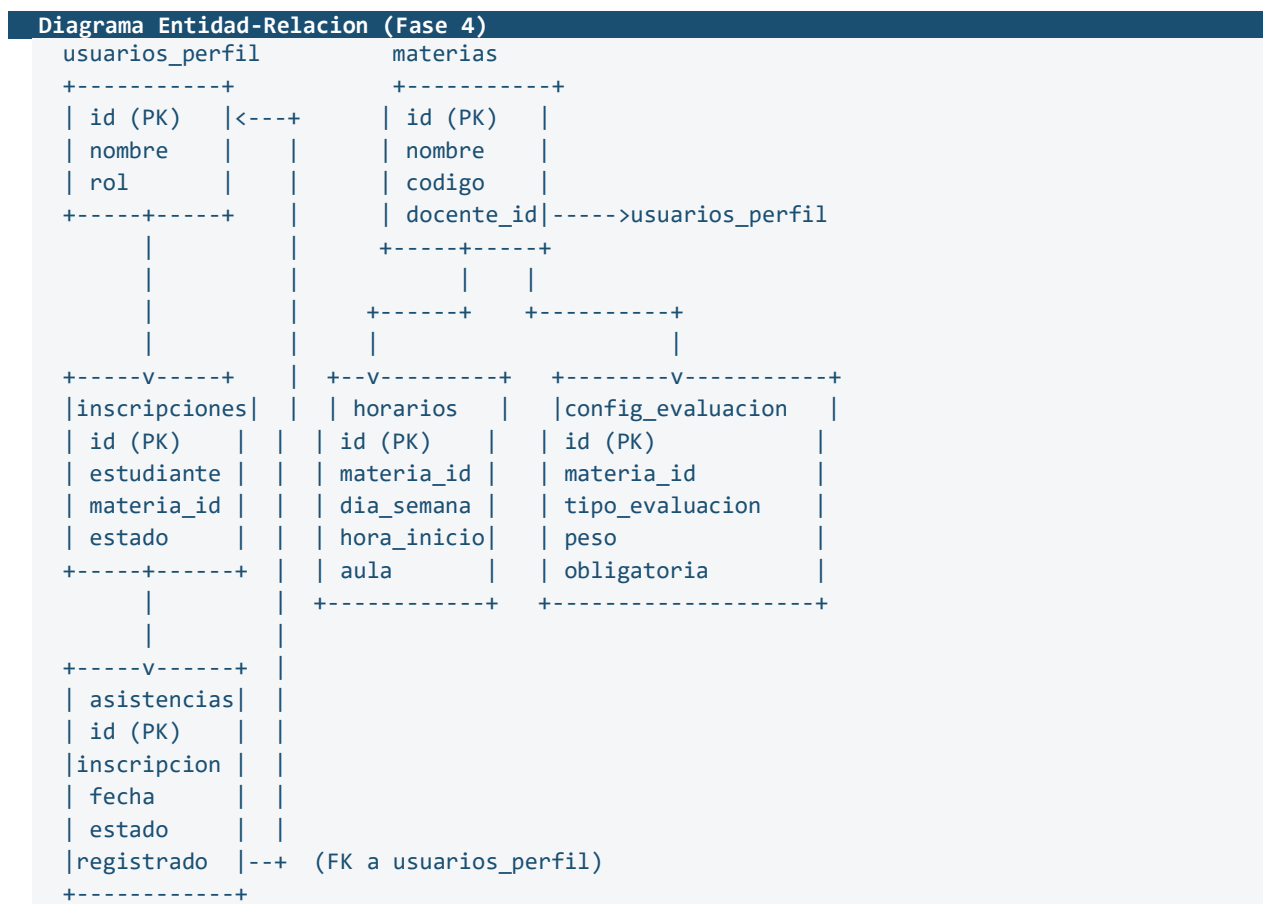
Prerrequisitos

- ✓ Entregas 1A a 7B completadas y validadas.
- ✓ Proyecto gestion-academica ejecutandose con npm run dev.
- ✓ Acceso al Supabase Dashboard (SQL Editor).
- ✓ Las 5 tablas del MVP existentes en la base de datos (usuarios_perfil, materias, inscripciones, calificaciones, documentos_academicos).
- ✓ Al menos un usuario con rol admin, un docente y un estudiante creados.

Teoría: Extendiendo el Esquema Relacional

En el MVP (Fases 1-3) trabajamos con 5 tablas que cubrían usuarios, materias e inscripciones. Ahora extendemos el esquema con 3 tablas que agregan funcionalidad operativa al sistema: la gestión de horarios, el registro de asistencias y la configuración de ponderaciones de evaluación.

Diagrama de Relaciones Extendido



Comparativa de las 3 Tablas Nuevas

Aspecto	horarios	asistencias	config_evaluacion
FK principal	materias	inscripciones	materias
Quien crea	Admin	Docente	Admin / Docente
Quien lee	Todos los roles	Docente + Estudiante	Todos los roles
Constraint clave	hora_fin > hora_inicio	UNIQUE(inscripcion, fecha)	peso BETWEEN 0-100
Trigger	No	Si (updated_at)	No
Registros típicos	2-3 por materia	1 por estudiante/día	4-7 por materia

NOTA: Relación indirecta

La tabla asistencias NO se vincula directamente a estudiante + materia. En su lugar, usa inscripcion_id como FK, lo cual garantiza que solo se puede registrar asistencia a estudiantes efectivamente inscritos en esa materia.

Concepto: CHECK vs Validación en Aplicación

Nivel	Ventaja	Ejemplo en este proyecto
CHECK (BD)	Garantía absoluta, imposible violar	dia_semana IN ('lunes',...), hora_fin > hora_inicio
Aplicación (API)	Lógica compleja, mensajes amigables	Suma de pesos = 100% por materia
Ambos	Defensa en profundidad	peso BETWEEN 0-100 (BD) + suma total (API)

TIP: Defensa en profundidad

La validación que la suma de pesos sea exactamente 100% se hará a nivel de la API Route (Entrega 10A), NO como trigger en PostgreSQL. Esto nos da mayor flexibilidad para configurar pesos parcialmente y validar al momento de guardar.

Paso 1: Crear la Tabla horarios

La tabla horarios define los bloques de clase de cada materia. Una materia puede tener múltiples bloques (por ejemplo, lunes de 8:00 a 9:30 y Miércoles de 10:00 a 11:30). Abre el SQL Editor en tu Supabase Dashboard y ejecuta el siguiente bloque:

```
SQL - Tabla horarios (ejecutar en Supabase SQL Editor)
-- =====
-- TABLA 6: Horarios
-- Define los bloques horarios de cada materia (dia, hora, aula).
-- =====

CREATE TABLE IF NOT EXISTS public.horarios (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  materia_id UUID NOT NULL
    REFERENCES public.materias(id) ON DELETE CASCADE,
  dia_semana TEXT NOT NULL
    CHECK (dia_semana IN (
      'lunes', 'martes', 'miercoles',
      'jueves', 'viernes', 'sabado'
    )),
  hora_inicio TIME NOT NULL,
  hora_fin TIME NOT NULL,
  aula TEXT,
  modalidad TEXT DEFAULT 'presencial' NOT NULL
    CHECK (modalidad IN (
      'presencial', 'virtual', 'hibrida'
    )),
  CONSTRAINT horario_valido CHECK (hora_fin > hora_inicio),
  UNIQUE(materia_id, dia_semana, hora_inicio)
);

-- Indices para consultas frecuentes
CREATE INDEX idx_horarios_materia
  ON public.horarios(materia_id);
CREATE INDEX idx_horarios_dia
  ON public.horarios(dia_semana);
CREATE INDEX idx_horarios_aula
  ON public.horarios(aula);
```

Análisis línea por línea

Línea / Clausula	Que hace	Por que importa
id UUID DEFAULT gen_random_uuid()	Genera un identificador unico automatico	Patron consistente con todas las tablas del proyecto
REFERENCES materias(id) ON DELETE CASCADE	FK a la tabla materias	Si se elimina una materia, sus horarios se borran automaticamente
CHECK (dia_semana IN (...))	Restringe a 6 valores validos	Evita errores como 'Lunes' (mayuscula) o 'domingo'
hora_inicio TIME, hora_fin TIME	Tipo TIME sin zona horaria	Almacena solo hora:minuto:segundo (ej: 08:00:00)
modalidad DEFAULT 'presencial'	Valor por defecto si no se especifica	La mayoría de clases en UCB son presenciales
CONSTRAINT horario_valido	CHECK que hora_fin > hora_inicio	Impide crear un bloque de 10:00 a 08:00
UNIQUE(materia_id, dia_semana, hora_inicio)	Restriccion compuesta	Una materia no puede tener 2 bloques el mismo día a la misma hora

ADVERTENCIA: Tipo TIME vs TIMESTAMPTZ

Usamos TIME (sin zona horaria) porque los horarios son locales a Bolivia (UTC-4). No necesitamos conversion de zonas. En cambio, created_at usa TIMESTAMPTZ porque es un timestamp absoluto.

Paso 2: Crear la Tabla asistencias

La tabla asistencias registra la presencia de cada estudiante por fecha. Se vincula a inscripciones (no directamente a estudiante + materia), lo cual garantiza integridad: solo puedes registrar asistencia a un estudiante inscrito.

SQL - Tabla asistencias (ejecutar en Supabase SQL Editor)

```
-- =====
-- TABLA 7: Asistencias
-- Registro de asistencia por estudiante por sesion de clase.
-- =====

CREATE TABLE IF NOT EXISTS public.asistencias (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  inscripcion_id UUID NOT NULL
    REFERENCES public.inscripciones(id) ON DELETE CASCADE,
  fecha DATE NOT NULL,
  estado TEXT NOT NULL
    CHECK (estado IN (
      'presente', 'ausente', 'justificado', 'tardanza'
    )),
  observaciones TEXT,
  registrado_por UUID NOT NULL
    REFERENCES public.usuarios_perfil(id),
  UNIQUE(inscripcion_id, fecha)
);

-- Indices
CREATE INDEX idx_asistencias_inscripcion
  ON public.asistencias(inscripcion_id);
CREATE INDEX idx_asistencias_fecha
  ON public.asistencias(fecha);
CREATE INDEX idx_asistencias_estado
  ON public.asistencias(estado);

-- Trigger para updated_at (reutiliza funcion del MVP)
CREATE TRIGGER trigger_updated_at_asistencias
  BEFORE UPDATE ON public.asistencias
  FOR EACH ROW
  EXECUTE FUNCTION public.actualizar_updated_at();
```

Análisis de decisiones de diseño

Decision	Alternativa descartada	Justificacion
FK a inscripciones	FK a estudiante + materia	Garantiza que solo inscritos tengan asistencia
estado con CHECK	BOOLEAN presente/ausente	4 estados permiten justificaciones y tardanzas
registrado_por UUID	Sin campo de auditor	Saber QUIEN registro la asistencia (trazabilidad)
UNIQUE(inscripcion, fecha)	Sin restriccion	Un estudiante solo tiene 1 registro por dia por materia
Trigger updated_at	Sin trigger	Reutilizamos la funcion del MVP (consistencia)

NOTA: Reutilizacion del Trigger

La funcion actualizar_updated_at() ya existe desde la Entrega 1B. El trigger simplemente la conecta a esta nueva tabla. Si intentas crear la funcion de nuevo, PostgreSQL dara un error de duplicado. Solo necesitas el CREATE TRIGGER.

TIP: Columna observaciones

Este campo TEXT es opcional (sin NOT NULL). Permite al docente agregar notas como 'Llego 15 min tarde por transporte' o 'Justificado por certificado medico'. Es crucial para la gestion academica real en UCB.

Paso 3: Crear la Tabla configuracion_evaluacion

Esta tabla define los pesos (ponderaciones) de cada tipo de evaluación por materia. Por ejemplo, en 'Programación I' el parcial_1 puede valer 20%, el parcial_2 vale 20%, la practica 30% y el proyecto 30%. Esto permite calcular la nota final ponderada automáticamente.

SQL - Tabla configuracion_evaluacion (ejecutar en Supabase SQL Editor)

```
-- =====
-- TABLA 8: Configuración de Evaluación
-- Ponderaciones de cada tipo de evaluación por materia.
-- =====

CREATE TABLE IF NOT EXISTS public.configuracion_evaluacion (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  materia_id UUID NOT NULL
    REFERENCES public.materias(id) ON DELETE CASCADE,
  tipo_evaluacion TEXT NOT NULL
    CHECK (tipo_evaluacion IN (
      'parcial_1', 'parcial_2', 'parcial_3',
      'final', 'practica', 'proyecto', 'extra'
    )),
  peso NUMERIC(5,2) NOT NULL
    CHECK (peso BETWEEN 0 AND 100),
  descripcion TEXT,
  obligatoria BOOLEAN DEFAULT true NOT NULL,
  UNIQUE(materia_id, tipo_evaluacion)
);

-- Indice
CREATE INDEX idx_config_eval_materia
  ON public.configuracion_evaluacion(materia_id);
```

Desglose del tipo NUMERIC(5,2)

Componente	Significado	Ejemplo
NUMERIC	Numero decimal de precision exacta	No pierde decimales como FLOAT
(5,2)	5 digitos totales, 2 decimales	Maximo: 999.99
CHECK peso BETWEEN 0 AND 100	Restriccion de rango	Rechaza -5 o 150

Componente	Significado	Ejemplo
Sin CHECK de suma	No validamos suma=100 en BD	Se valida en la API Route (Entrega 10A)

ADVERTENCIA: Por que no validar la suma en PostgreSQL?

Un CHECK constraint opera sobre UNA FILA. Para validar que la suma de pesos de todas las filas de una materia sea 100, necesitaríamos un TRIGGER o una funcion PL/pgSQL. Decidimos hacerlo en la API Route por simplicidad pedagogica y mayor control sobre los mensajes de error.

Ejemplo de configuracion tipica

tipo_evaluacion	peso	descripcion	obligatoria
parcial_1	20.00	Primer examen parcial	true
parcial_2	20.00	Segundo examen parcial	true
practica	30.00	Laboratorios y talleres	true
proyecto	30.00	Proyecto final integrador	true
extra	0.00	Puntos adicionales	false

En este ejemplo, la suma es **100.00** (sin contar 'extra' con peso 0). La columna obligatoria indica si el estudiante debe tener nota en ese rubro para aprobar.

Paso 4: Configurar Politicas RLS

Ahora habilitamos RLS en las 3 tablas nuevas y creamos politicas diferenciadas por rol. Recuerda que sin politicas RLS habilitadas, Supabase bloquea TODAS las consultas con la Publishable Key.

4.1 - RLS para horarios

SQL - RLS horarios

```
-- Habilitar RLS
ALTER TABLE public.horarios ENABLE ROW LEVEL SECURITY;

-- Lectura: cualquier usuario autenticado puede ver horarios
CREATE POLICY "horarios_select_autenticado"
ON public.horarios FOR SELECT
TO authenticated
USING (true);

-- Insercion: solo admin
CREATE POLICY "horarios_insert_admin"
ON public.horarios FOR INSERT
TO authenticated
WITH CHECK (
  EXISTS (
    SELECT 1 FROM public.usuarios_perfil
    WHERE id = auth.uid() AND rol = 'admin'
  )
);

-- Actualizacion: solo admin
CREATE POLICY "horarios_update_admin"
ON public.horarios FOR UPDATE
TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.usuarios_perfil
    WHERE id = auth.uid() AND rol = 'admin'
  )
);

-- Eliminacion: solo admin
CREATE POLICY "horarios_delete_admin"
ON public.horarios FOR DELETE
TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.usuarios_perfil
```

```

        WHERE id = auth.uid() AND rol = 'admin'
    )
);

```

4.2 - RLS para asistencias

SQL - RLS asistencias

```

-- Habilitar RLS
ALTER TABLE public.asistencias ENABLE ROW LEVEL SECURITY;

-- Lectura para docentes: solo asistencias de SUS materias
CREATE POLICY "asistencias_select_docente"
ON public.asistencias FOR SELECT
TO authenticated
USING (
    EXISTS (
        SELECT 1 FROM public.inscripciones i
        JOIN public.materias m ON m.id = i.materia_id
        WHERE i.id = asistencias.inscripcion_id
        AND m.docente_id = auth.uid()
    )
);

-- Lectura para estudiantes: solo SUS propias asistencias
CREATE POLICY "asistencias_select_estudiante"
ON public.asistencias FOR SELECT
TO authenticated
USING (
    EXISTS (
        SELECT 1 FROM public.inscripciones i
        WHERE i.id = asistencias.inscripcion_id
        AND i.estudiante_id = auth.uid()
    )
);

-- Lectura para admin: todas las asistencias
CREATE POLICY "asistencias_select_admin"
ON public.asistencias FOR SELECT
TO authenticated
USING (
    EXISTS (
        SELECT 1 FROM public.usuarios_perfil
        WHERE id = auth.uid() AND rol = 'admin'
    )
);

-- Insercion: solo docentes de la materia correspondiente

```

```

CREATE POLICY "asistencias_insert_docente"
ON public.asistencias FOR INSERT
TO authenticated
WITH CHECK (
    EXISTS (
        SELECT 1 FROM public.inscripciones i
        JOIN public.materias m ON m.id = i.materia_id
        WHERE i.id = inscripcion_id
        AND m.docente_id = auth.uid()
    )
);

-- Actualizacion: solo el docente que registro (o admin)
CREATE POLICY "asistencias_update_docente"
ON public.asistencias FOR UPDATE
TO authenticated
USING (
    registrado_por = auth.uid()
    OR EXISTS (
        SELECT 1 FROM public.usuarios_perfil
        WHERE id = auth.uid() AND rol = 'admin'
    )
);

```

NOTA: Políticas OR en asistencias

La tabla asistencias tiene 3 políticas SELECT separadas (docente, estudiante, admin). PostgreSQL evalúa todas las políticas con OR: si CUALQUIERA devuelve true, el registro es visible. Esto permite que cada rol vea exactamente lo que le corresponde.

4.3 - RLS para configuracion_evaluacion

SQL - RLS configuracion_evaluacion

```

-- Habilitar RLS
ALTER TABLE public.configuracion_evaluacion
    ENABLE ROW LEVEL SECURITY;

-- Lectura: cualquier usuario autenticado
CREATE POLICY "config_eval_select_autenticado"
ON public.configuracion_evaluacion FOR SELECT
TO authenticated
USING (true);

-- Insercion: admin o docente de la materia
CREATE POLICY "config_eval_insert_admin_docente"
ON public.configuracion_evaluacion FOR INSERT
TO authenticated

```

```

WITH CHECK (
  EXISTS (
    SELECT 1 FROM public.usuarios_perfil
    WHERE id = auth.uid() AND rol = 'admin'
  )
  OR EXISTS (
    SELECT 1 FROM public.materias
    WHERE id = materia_id
    AND docente_id = auth.uid()
  )
);

-- Actualizacion: admin o docente de la materia
CREATE POLICY "config_eval_update_admin_docente"
ON public.configuracion_evaluacion FOR UPDATE
TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.usuarios_perfil
    WHERE id = auth.uid() AND rol = 'admin'
  )
  OR EXISTS (
    SELECT 1 FROM public.materias
    WHERE id = materia_id
    AND docente_id = auth.uid()
  )
);

-- Eliminacion: solo admin
CREATE POLICY "config_eval_delete_admin"
ON public.configuracion_evaluacion FOR DELETE
TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.usuarios_perfil
    WHERE id = auth.uid() AND rol = 'admin'
  )
);

```

Paso 5: Verificar las Politicas RLS

Despues de ejecutar todos los bloques SQL, verifica en el Supabase Dashboard que cada tabla tenga RLS habilitado y las politicas correctas. Ve a Authentication > Policies en el panel lateral.

Resumen de Politicas por Tabla

Tabla	SELECT	INSERT	UPDATE	DELETE
horarios	Autenticado	Admin	Admin	Admin
asistencias	Docente(sus mat) / Estudiante(sus) / Admin	Docente(sus mat)	Registrador / Admin	---
config_evaluacion	Autenticado	Admin / Docente(su mat)	Admin / Docente(su mat)	Admin

IMPORTANTE: asistencias no tiene DELETE

Intencionalmente NO creamos politica DELETE para asistencias. Un registro de asistencia no debe eliminarse una vez creado. Si el docente comete un error, usa UPDATE para corregir el estado. Esto garantiza trazabilidad y auditoria.

Paso 6: Actualizar la Navegacion del Sidebar

Ahora agregamos las nuevas secciones al sidebar. Abre el archivo lib/navigation.ts y agrega las entradas correspondientes para cada rol. Recuerda que este archivo ya existe desde la Entrega 3A.

TypeScript - lib/navigation.ts (actualizar)

```
// Agregar los nuevos imports de iconos en la parte superior
import {
  Home, BookOpen, ClipboardList, User, Users,
  GraduationCap, Calendar, CheckSquare, Settings,
  Clock,           // <-- NUEVO: para horarios
  UserCheck,       // <-- NUEVO: para asistencias
  BarChart3,       // <-- NUEVO: para config evaluacion
} from "lucide-react";

// ... (tipos existentes sin cambios) ...

// Actualizar la configuracion por rol:

// — ADMIN —
// Agregar al array de items del admin:
{
  title: "Horarios",
  url: "/admin/horarios",
  icon: Clock,
},
{
  title: "Asistencias",
  url: "/admin/asistencias",
  icon: UserCheck,
},
{
  title: "Evaluacion",
  url: "/admin/evaluacion",
  icon: BarChart3,
},

// — DOCENTE —
// Agregar al array de items del docente:
{
  title: "Horarios",
  url: "/docente/horarios",
  icon: Clock,
},
{
  title: "Asistencias",
  url: "/docente/asistencias",
}
```



```
    icon: UserCheck,
  },
  {
    title: "Evaluacion",
    url: "/docente/evaluacion",
    icon: BarChart3,
  },
  // — ESTUDIANTE —
  // Agregar al array de items del estudiante:
  {
    title: "Horarios",
    url: "/estudiante/horarios",
    icon: Clock,
  },
  {
    title: "Asistencias",
    url: "/estudiante/asistencias",
    icon: UserCheck,
  },
}
```

TIP: Estudiantes NO ven Evaluacion en el sidebar

Los estudiantes no configuran ponderaciones. Ellos verán los pesos de evaluación dentro de la vista de cada materia (como información de consulta), pero no necesitan una sección dedicada en el sidebar.

Paso 7: Crear Paginas Placeholder

Crea las nuevas paginas como placeholders. Estas se implementaran completamente en las entregas 8B a 10B. Sigue el mismo patron de las paginas placeholder de la Entrega 3B.

Bash - Crear directorios

```
# Desde la raiz del proyecto gestion-academica/

# Admin
mkdir -p app/(dashboard)/admin/horarios
mkdir -p app/(dashboard)/admin/asistencias
mkdir -p app/(dashboard)/admin/evaluacion

# Docente
mkdir -p app/(dashboard)/docente/horarios
mkdir -p app/(dashboard)/docente/asistencias
mkdir -p app/(dashboard)/docente/evaluacion

# Estudiante
mkdir -p app/(dashboard)/estudiante/horarios
mkdir -p app/(dashboard)/estudiante/asistencias
```

Plantilla para cada page.tsx

Usa esta plantilla para cada pagina placeholder. Solo cambia el titulo y la descripcion:

TypeScript - Ejemplo: app/(dashboard)/admin/horarios/page.tsx

```
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Clock } from "lucide-react";

export default function HorariosAdminPage() {
  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">
        Gestion de Horarios
      </h1>
      <Card>
        <CardHeader>
          <CardTitle className="flex items-center gap-2">
            <Clock className="h-5 w-5" />
            Horarios (Proximamente)
          </CardTitle>
        </CardHeader>
```

```

        <CardContent>
          <p className="text-muted-foreground">
            Esta seccion se implementara en la Entrega 8B.
          </p>
        </CardContent>
      </Card>
    </div>
  );
}

```

Lista de 8 paginas a crear

Ruta	Componente	Titulo	Icono
admin/horarios/page.tsx	HorariosAdminPage	Gestion de Horarios	Clock
admin/asistencias/page.tsx	AsistenciasAdminPage	Asistencias (General)	UserCheck
admin/evaluacion/page.tsx	EvaluacionAdminPage	Configuracion de Evaluacion	BarChart3
docente/horarios/page.tsx	HorariosDocentePage	Mis Horarios	Clock
docente/asistencias/page.tsx	AsistenciasDocentePage	Registrar Asistencias	UserCheck
docente/evaluacion/page.tsx	EvaluacionDocentePage	Ponderaciones	BarChart3
estudiante/horarios/page.tsx	HorariosEstudiantePage	Mi Horario	Clock
estudiante/asistencias/page.tsx	AsistenciasEstudiantePage	Mis Asistencias	UserCheck

Verificacion Final

Antes de continuar con la Entrega 8B, verifica cada punto:

- ✓ Las 3 tablas aparecen en Supabase Dashboard > Table Editor: horarios, asistencias, configuracion_evaluacion.
- ✓ Cada tabla tiene RLS habilitado (candado verde en Table Editor).
- ✓ Los indices aparecen en la pestana SQL: idx_horarios_materia, idx_asistencias_inscripcion, idx_config_eval_materia, etc.
- ✓ El trigger trigger_updated_at_asistencias esta creado (verificar en Database > Triggers).
- ✓ La navegacion del sidebar muestra las nuevas secciones para cada rol.
- ✓ Las 8 paginas placeholder cargan correctamente al navegar.
- ✓ npm run dev funciona sin errores en la consola.
- ✓ Los horarios se vinculan correctamente a materias existentes (probar INSERT manual).

TIP: Prueba rapida con INSERT manual

En el SQL Editor de Supabase, prueba insertar un horario con: `INSERT INTO horarios (materia_id, dia_semana, hora_inicio, hora_fin, aula) VALUES ('<uuid-de-una-materia>', 'lunes', '08:00', '09:30', 'Aula 201');` Si funciona, el esquema esta correcto.

Ejercicio Propuesto

Completa estas 4 tareas para reforzar lo aprendido:

1. Tarea 1: Inserta 3 bloques horarios para una misma materia (Lunes, Miercoles y Viernes) y verifica que el constraint UNIQUE impida duplicados.
2. Tarea 2: Intenta insertar un horario con hora_fin menor que hora_inicio. Observa el mensaje de error de PostgreSQL y anota que constraint lo bloquea.
3. Tarea 3: Crea una configuracion de evaluacion completa para una materia (parcial_1: 25%, parcial_2: 25%, practica: 25%, proyecto: 25%) e intenta agregar un duplicado de 'parcial_1' para verificar el UNIQUE.
4. Tarea 4: Conectate como estudiante (usando la Publishable Key) e intenta hacer un INSERT en la tabla horarios. Verifica que la politica RLS lo bloquee con un error de permisos.

Proximo: Entrega 8B

En la Entrega 8B implementaremos el modulo completo de Horarios: API Routes (CRUD para admin, GET para docente y estudiante), componentes de interfaz con tabla y formulario Dialog, y la vista semanal tipo grilla para visualizar los horarios.

Que vamos a construir	Archivos nuevos
API Route: GET + POST horarios (admin)	app/api/admin/horarios/route.ts
API Route: PUT + DELETE horario (admin)	app/api/admin/horarios/[id]/route.ts
API Route: GET horarios docente	app/api/docente/horarios/route.ts
API Route: GET horarios estudiante	app/api/estudiante/horarios/route.ts
Componente: tabla + dialog CRUD	components/dashboard/admin/horarios-table.tsx
Componente: vista semanal	components/dashboard/horario-semanal.tsx
Schema Zod de validacion	lib/validations/horario.ts